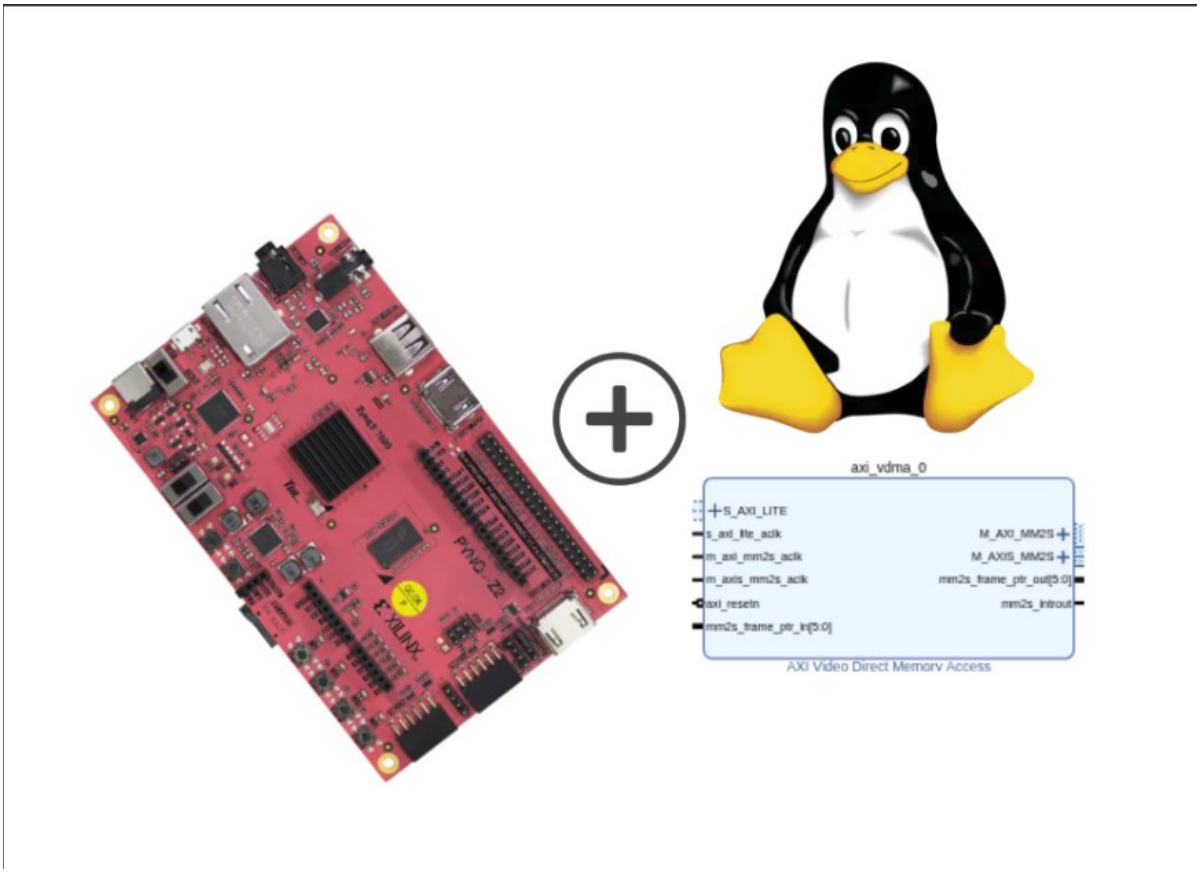


PYNQ-Z2 EMBEDDED GRAPHICS

Oliwier Jaworski
PXL hogeschool
Elektronica ICT
oliwier.jaworski@student.pxl.be

ABSTRACT

quick intro explain what this whole thing is about

Keywords: Zuboard-1CG, Petalinux, DPU, AXI-interfaces, embedded linux, onnx, model quantization

Table of Contents

1. INTRODUCTION....3
2. HARDWARE REQUIREMENTS....3
3. HARDWARE SETUP....3
3.1 INSTALLATION PREREQUISITS....3
3.2 VIVADO PROJECT....3
3.2.1 TCL CANT FIND BOARDFILES....3
4. CREATING THE LINUX IMAGE....4
4.1 PROJECT CREATION....4
4.2 KERNEL CONFIGURATION....4
4.3 PETALINUX CONFIGURATION....4
4.4 ADDING USER PACKAGES....4
4.5 ROOFS CONFIGURATION....5
4.6 DEVICE TREE TWEAKS....6
4.7 BUILDING PROJECT....6
5. PACKAGING CUSTOM LINUX....6
5.1 IMAGE FLASHING....7
6. LOADABLE KERNEL MODULE....7
6.1 APPROACHES TO MODULES....7
6.1 ENVIRONMENT SETUP....7
6.2 CREATING A HELLO-WORLD MODULE....7
6.3 DEPLOYING KERNEL MODULES....8
6.3 VDMA-OVERLAY WALKTHROUGH....8
6.3 VDMA OVERLAY MODULE....9
6. VALIDATION....9
6. DEPLOYMENT....9
REFERENCES....10

1. INTRODUCTION

This project provides a step-by-step guide for setting up a custom PetaLinux image on the PYNQ-Z2 development board and implementing a Video Direct Memory Access (VDMA) pipeline using loadable kernel modules.

The PYNQ-Z2 is a Xilinx Zynq-7000 SoC-based board that combines a dual-core ARM Cortex-A9 processor with programmable FPGA logic, making it well suited for embedded video processing applications. This guide covers the full development flow — from Vivado hardware design and PetaLinux image creation, through to packaging, flashing, and deploying a custom VDMA kernel module on the target hardware.

2. HARDWARE REQUIREMENTS

The list below describes the minimum hardware and software requirements.

- Host machine using a compatible linux distribution(ubuntu 20.04)
- Vivado (2024.1)
- Vitis (2024.1)
- Petalinux (2024.1, codename: nanbield)
- PYNQ-Z2
- Ethernet and USB-C data cable
- SD-card(>8GB)

3. HARDWARE SETUP

In this chapter the user will recreate the Zynq +VDMA block-design using the tcl-script and tcl utility, after which the XSA(Xilinx Support Archive) will then be used for hardware development.

3.1 INSTALLATION PREREQUISITS

1. Start by downloading the bdf-files(board definition files) for the PYNQ-Z2. After which copy the folder over to the following directory:

```
<PATH>Vivado/2024.1/data/xhub/boards/
XilinxBoardStore/boards/Xilinx/
```

- prefix <PATH> will depend on where vivado is installed.

2. Download the Digilent vivado ip zip-file from the github release page.
3. Inside the folder take note of the path to the “ip” folder:

3.2 VIVADO PROJECT

1. Launch the vivado environment.
   - On **windows** open a vivado tcl command window(2024.2)
   - On **Linux** source the vivado settings64.sh script using the following command:

```
source <PATH-TO-VIVADO-INSTALL>/settings64.sh
```

2. Build the vivado project using the following command:

```
vivado -mode batch -source script.tcl
```

   - This command will generate the project automatically.

!Note: section 3.2.1 TCL CANT FIND BOARDFILES explains the following; If errors occur during initialization (e.g., Vivado cannot find the correct BDF files), this is likely due to a version mismatch between the Tcl script and your installed board files.

3. Open the project inside Vivado and add the Digilent IP repository.
   - This step is required; otherwise, errors will occur when creating the HDL wrapper.
4. Finalize the design
   - Open the block design in Vivado.
   - Create the HDL wrapper.
   - Generate the bitstream.
   - Export the hardware design including the bitstream.
5. Save the XSA-file
   - During export, Vivado will generate an .xsa file. Make sure to note the location where this file is saved, as it will be required in later steps.

3.2.1 TCL CANT FIND BOARDFILES

Open the board.xml file located at:

```
<PATH>Vivado/2024.1/data/xhub/boards/XilinxBoardStore/
boards/Xilinx/zub1cg/1.2/board.xml
```

From this file, note:

- file_version
- vendor
- name

Then, in the Tcl script, locate the **"Set project properties"** section and update the hardcoded board parameters to match these values.

4. CREATING THE LINUX IMAGE

The following steps outline the process of creating a embedded-linux image for the compatible platform of choice. This image will contain the user generated .xsa hardware logic. It also contains device drivers and interfaces for the user to be able to interact with the hardware.

To complete this process, the user must have access to an SD card and a method to flash the generated image onto it.

!Note: Most troubleshooting tends to occur in this section due to the frequent changes, patches, and updates within the toolchain and ecosystem. If your results do not exactly match what is shown in this guide, it may not necessarily mean the steps were followed incorrectly. However, identifying the issue may require additional investigation and effort. Some of the most common pitfalls are outlined later in this section.

4.1 PROJECT CREATION

1. Source both the petalinux and vivado environment
   - Deactivate any environment which may influence linux paths such as conda, python venv, etc...
   - Problem might occur if vivado is not sourced
   - this environment is only local to that console instance.
2. Use the following command to create your petalinux project.

```
petalinux-create -t project -n my_project --template
zynq --silentconfig
```

3. Add the hardware design(XSA) to the project

```
petalinux-config
–get-hw-description=/home/<path>/custom/file.xsa
```

4. Changing OE terminal

   This prevents the next step from hanging up on 99% during package building on some systems.

   - location:

```
<petalinux-project>/project-spec/meta-user/conf/
petalinuxbsp.conf
```

   - replace the commented out line with:

```
OE_TERMINAL = "xterm"
```

4.2 KERNEL CONFIGURATION

- Command:

```
petalinux-config -c kernel
```

The bullet points below outline the paths and features that must be enabled (<*>) or disabled (< >).

➢ Device Drivers → Graphics support
- <*>Enable Legacy fbdev support for your modesetting driver
- <*>Simple framebuffer driver

➢ Device Drivers → Graphics support → Frame buffer Devices
- <*>support for frame buffer device drivers
- <*>Provide legacy /dev/fb* device

➢ Device Drivers → USB support
- <*>USB announce new devices
- < > USB Gadget Support

4.3 PETALINUX CONFIGURATION

- Command:

```
petalinux-config
```

➢ Image Packaging Configuration
- Root filesystem type(EXT4 (SD/eMMC/SATA/USB)

4.4 ADDING USER PACKAGES

by adding packages before the linux image is created the user can expand functionality that's provided with the image.

The **Openembedded** ecosystem provides the packages the user can import into the linux image, by either using the bitbake recipe or a full layer. For full layers the procedure is not explained in this guide.

the collection of packages can be found in the references section. By going to the "Openembedded layer index" link.

For this project only the following recipes were added:

- **xf86-video-fbdev 0.5.0**

  This package provides a driver for framebuffer devices

- **libsdl2 2.32.10**

  This package is optional

*Note: Some packages will cause the **petalinux build process** to fail. Each package has a "patches" section which will contain possible fixes. Which may or may not fix the issue.*

In some cases one version will work while an other will fail. The only way to go about it is through trail and error.

The following steps explain the user how to add a recipe to the petalinux project.

1. Visit the "**openembedded layer index**" webpage"(found in references) or visit one of the reference links which will directly point you to the recipe used in this guide.
2. Find a section called "recipe file" and open it. This contains the files and layout which will be used.
3. Take note of the "path" string in the tree section which will look something like the following for a "one file recipe":

```
path:
root/meta/recipes-graphics/libsdl2_2.32.10.bb
```

4. Inside the petalinux project navigate to:

```
<project>/project-spec/meta-user/
```

5. from the found path only create the path after meta, because we will be staying in the meta-user layer. Which will look as following:

```
<project>/project-spec/meta-user/recipes-
graphics/libsdl2
```

6. inside the libsdl2 folder create the bitbake(.bb) file and copy the content from the served <recipe>.bb into that file.

   <u>Note:</u> in a multi-folder recipe make sure to copy all the files and paths. And add the patches with the same method provided.

   The following is a directory tree of the added recipes:

```
project-spec/meta-user/recipes-graphics/
├── libsdl2
│   └── libsdl2_2.32.10.bb
└── xorg-driver
    ├── xf86-video-fbdev_0.5.0.bb
    ├── xorg-driver-common.inc
    └── xorg-driver-video.inc
```

   <u>Note:</u> The used recipes can be found in the github repository in the following path:

```
<repo>/petalinux-yocto/linux-images/
guide_VDMA_HDMI
```

7. The .bb files provide the name by which the recipe will be included in the petalinux project. By taking note of the filename for example libsdl2_2.32.10.10.bb is found as "libsdl2" by petalinux.
8. Include the recipes names in the following file, for petalinux-rootfs to recognise them:

```
<project>/project-spec/meta-user/conf/user-
rootfsconfig
```

   They should be included with a "CONFIG_" prefix as following:

```
CONFIG_libsdl2
CONFIG_xf86-video-fbdev
```

4.5 ROOFS CONFIGURATION

5. Configuring the root file system

- Command:

```
petalinux-config -c rootfs
```

The bullet points below outline the paths and features that must be enabled ([*]) or disabled ([ ]).

➢ User Packages

- [*]libsdl2
- [*]xf86-video-fbdev

➢ Image Features

- [*]debug-tweaks

➢ Petalinux Package Groups → matchbox

- [*] packagegroup-petalinux-matchbox

➢ Petalinux Package Groups → packagegroup-petalinux-x11

- [*] packagegroup-petalinux-x11

➢ Petalinux Package Groups → packagegroup-petalinux-qt

- [*] packagegroup-petalinux-qt
- [*] populate_sdk_qt5

➢ Filesystem Packages → x11 → base → xserver-xorg

- [*] xserver-xorg

➢ Filesystem Packages → x11 → base → xserver-xf86-config

- [*] xserver-xf86-config

➢ Filesystem Packages → x11 → base → libs → libmatchbox

- [*] libmatchbox

➢ Filesystem Packages → x11 → base → libs → xkeyboard-config

- [*] xkeyboard-config

➢ Filesystem Packages → x11 → matchbox-keyboard

- [*] matchbox-keyboard

➢ Filesystem Packages → x11 → matchbox-session

- [*] matchbox-session

➢ Filesystem Packages → x11 → mini-x-session

- [*] mini-x-session

➢ Filesystem Packages → x11 → pcmanfm

- [*] pcmanfm

➢ Filesystem Packages → x11 → utils → matchbox-terminal

- [*] matchbox-terminal

➢ Filesystem Packages → x11 → wm → matchbox-wm

- [*] matchbox-wm

➢ Filesystem Packages → x11 → xserver-nodm-init

- [*] xserver-nodm-init

4.6 DEVICE TREE TWEAKS

The device tree is currently missing the reserved memory for vdma read/write(frame-buffer), usb and vdma kernel driver entries to add these edit the following file located in:

```
project-spec/meta-user/recipes-bsp/device-tree/
files/system-user.dtsi
```

The used device-tree.bbappend can be found in the github repository under the following prefix:

```
petalinux-yocto/linux-images/guide_VDMA_HDMI
```

followed by the path already mentioned earlier.

----→ what was changed????

4.7 BUILDING PROJECT

```
petalinux-build
```

- if changes are made which need full rebuild use the following command and then build the project again.

```
petalinux-build -x mrproper
```

5. PACKAGING CUSTOM LINUX

After the build completes, a new directory is created inside the PetaLinux project at **/images/linux/**. This directory contains all files required to create the boot image and package the Linux distribution into a .**wic** file. The resulting .**wic** file can then be written to the SD card using a tool such as **Balena Etcher**, or alternatively by manually copying the relevant files to the appropriate SD card partitions.

1. Generation of boot image(**BOOT.BIN**)

- The boot image (**BOOT.BIN**) typically contains the **First Stage Boot Loader (FSBL)**, which initializes the processing system (DDR, clocks, and I/O) and then loads the **Second Stage Boot Loader (U-Boot)**. U-Boot provides the environment to load and start the Linux kernel.
- Optionally, the boot image can also include the FPGA bitstream to configure the programmable logic. In this tutorial, the bitstream (**system.bit**) is included.
- The **BOOT.BIN** file is generated inside the **images/linux/** directory, as the command structure and available arguments make this the most convenient working location.

- The following command must be executed from the image/linux directory of the **PetaLinux project**; otherwise, the files will not be found.

```
petalinux-package boot --fsbl zynq_fsbl.elf --fpga
system.bit --u-boot u-boot.elf
```

- The following command creates a .wic image

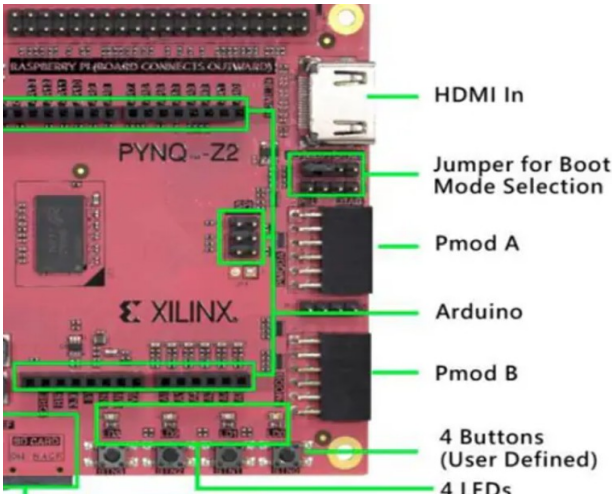
```
petalinux-package wic -b "BOOT.BIN, uImage,
boot.scr"
```

Together, these steps form the minimal setup required for booting the custom Linux system from the SD card.

5.1 IMAGE FLASHING

To flash the image onto the **SD** card, **Balena Etcher** was used. Simply select the generated **.wic** file from **/images/linux/**, choose the target SD card, and click Flash. Wait for the process to complete before removing the card.

Note: Ensure the boot mode jumper is set to SD rather than **JTAG**; otherwise, the board will not boot from the SD card.



6. LOADABLE KERNEL MODULE

The custom Linux image does not include a driver required to configure the AXI VDMA. To address this limitation, a custom Linux driver was developed based on a VDMA test driver provided by Xilinx. The functionality of this driver is outlined in the following section, along with the procedures for its setup, compilation, and deployment within a PetaLinux image.

6.1 APPROACHES TO MODULES

A kernel module can be developed using one of the following approaches:

- Use the **petalinux-build module** system

  This approach is documented in the PetaLinux Tools documentation and provides a structured and user-friendly method for creating Linux kernel modules. It offers a predefined template design as a starting point, along with an integrated Petalinux build interface. This simplifies the development process, particularly for users who are new to kernel module development, and ensures compatibility with the overall PetaLinux project configuration.

- Out of tree modules with **artifacts** by **petalinux-build**

  This method offers faster compilation and more flexibility during development. The developer creates and maintains their own module template and compiles it using standard make tools rather than the PetaLinux build system.

  A drawback of this approach is that compatibility issues may arise when attempting to integrate or deploy these modules later using the petalinux-build module system.

6.1 ENVIRONMENT SETUP

Using **petalinux-build module** system

1. Navigate to the **petalinux project** directory
2. use the following command:

```
petalinux-create modules -n module-name –enable
```

- -enable: module will be enabled at boot.

which will create an entry at:

```
"project-spec/meta-user/recipes-modules"
```

Using Out of tree modules with **artifacts** by **petalinux-build**

1. Build the petalinux-project atleast once, for the necessary files to be generated.
2. Navigate to the following **petalinux project** directory

```
/build/tmp/work-shared/zynq-generic-7z020/kernel-build-artifacts
```

3. copy the following files:
   - Module.symvers
   - .config
4. and add them to the following relative directory:

```
../kernel-source
```

5. export this path

```
export KERNEL_SRC=$PWD
```

Note: The user must export this path each time a new terminal session is started. This can be avoided by adding the export command to the .bashrc file.

6.2 CREATING A HELLO-WORLD MODULE.

First to verify the environment is set up correctly, we will deploy a dummy "hello-world!' module onto the device. depending on your approach either:

- Build the single module using petalinux-build command.(this may take some time if its the first build.)

```
Petalinux-build -c module-name
```

- compile the module using make and provided template files on github.

1. Find the template design:

```
<github-repo>/petalinux-yocto/apps/kernel-hello-world/
```

2. compile using the following command:

```
make all
```

6.3 DEPLOYING KERNEL MODULES

To deploy the module onto the hardware we simply copy the generated "<file>.ko" onto the hardware.

This file can be found in the following directory depending on approach:

- **petalinux-build** approach

```
find ./ -type f -name "*<module>.ko"
```

Note: "<module>" is the name given to the file during module creation e.g "-n <module-name>"

- **out of tree cmake** approach:

```
<kernel-module-source-directory>/<module>.ko
```

copy this file to the device and use the following command to insert it:

```
insmod <modfile>.ko
```

hardware and use the following command to remove it:

```
rmmod <modfile>.ko
```

The module implements both a probe and a remove function, each of which writes a message to the kernel log (dmesg). To view the output, use the following command to display the last 20 log entries:

```
dmesg | tail -20
```

6.4 VDMA-OVERLAY WALKTHROUGH

This section will quickly go through the vmda-overlay module's code created which sets up the axi-vdma device.

This module can be found in the github repo in the following directory:

```
<github-repo>/project-spec/meta-user/recipes-modules/vdma-overlay/files
```

1. **Headers** & **Module** Metadata:
   - Includes Linux kernel headers for: dma, device tree and platform driver support.
   - Defines module info such as license, author, description and version
2. **Macros** and **constants**
   - framebuffer configuration
   - derived values
   - frm_cnt(number of framebuffers)
3. Data structures
   - **rmem_fb_info**: holds reserved framebuffer info fields
   - **vdmaOverlay_info:** main driver state container, stores device handles and info fields
4. **vdmaOverlay_configure**
   - Configures VDMA channel: genlock, master mode, frame counting, reset
   - Maps framebuffer memory for DMA

- Prepares interleaved DMA transfers for each frame
- Submits and starts DMA

5. **rmem_fb_check**
   - Verifies /reserved-memory node in device tree
   - Finds framebuffer node and reads reg property
   - Extracts physical address and size
   - Logs framebuffer information
6. **vdmaOverlay_probe**
   - Allocates driver state
   - Validates framebuffer and checks memory size
   - Reads xlnx,num-fstores property
   - Requests DMA channel
   - Configures VDMA
   - Stores driver data with platform_set_drvdata
   - Handles errors and cleanup
7. vdmaOverlay_remove
   - Synchronizes and terminates DMA
   - Unmaps all DMA buffers
   - Releases DMA channel
   - Frees allocated driver state
8. Device Tree Matching
   - Matches devices compatible with "xlnx,vdma-overlay-1.0"
9. Platform Driver Registration
   - Defines platform_driver with .probe and .remove
   - Registers driver in module_init and unregisters in module_exit

6.5 DEPLOYING OUT OF TREE KERNEL MODULES

This section is intended only for users developing modules using the out-of-tree approach.

To include the module in the system image, you must create it following the PetaLinux module build procedure, replacing the default <module>.c content with your own code. Some adjustments may be necessary to resolve compatibility issues or other conflicts.

6.6 PETALINUX TROUBLESHOOTING

If kernel module builds fail with optimization-related errors, add the following line to the module's **Makefile**:

```
KBUILD_CFLAGS_MODULE += -O1
```

This forces the compiler to use lower optimization (-O1) for the module, preventing miscompilations caused by aggressive optimizations.

7. APP DEPLOYMENT

This section will contain user code written for gui using qt framework aswell as a small template sdl2 app which are deployed on the pynq-2 petalinux image.

7.1 ENVIRONMENT SETUP

The cross-compilation tools are required to build native applications for the target platform. These tools include all necessary libraries, toolchains, and utilities for development.

1. To generate the SDK use the following command from within the petalinux project:

```
petalinux-build --sdk
```

2. the generated sdk can be found in:

```
images/linux
```

3. execute the sdk script and install the sdk to the folder of your choice.
4. Once installed, before compiling any applications, source the environment setup script which can be found in the sdk installation directory:

```
source environment-setup-aarch64-xilinx-linux
```

5. After this, the cross-compilation tools and libraries are ready for building native applications for the target platform.

7.2 VALIDATION

The github repository provides applications and their sources which target the cross compilation toolchain. Simply run "**make default**" this will create a **"<filename>.ELF".** This file is the binary output which can run on the device.

The repository path to the example applications is:

```
petalinux-yocto/apps
```

<u>Note:</u> the "kernel"-prefix means its a kernel driver these are not meant to be compiled using the cross compilation toolchain but with the earlier provided steps.

7.2 DEPLOYING EXECUTABLE

*The github repository provides applications and their sources which target the cross compilation toolchain. Simply run "**make default**" this will create a **"<filename>.ELF".** This file is the binary output which can run on the device.*

By using a tool such as "scp" the user can transfer the file to the device.

7.2 VALIDATION QT-APP

I was able to successfully cross-compile a Qt application, but the original tutorial used is no longer available. The only remaining reference is the Qt project located at:

```
petalinux-yocto/apps/qt_app
```

This project can serve as a useful reference, particularly because the `CMakeLists.txt` file previously caused multiple errors. The configuration included may help identify and resolve similar issues more quickly when working with CMake, making it easier to diagnose problems during the build process.

REFERENCES

2.0: HARDWARE REQUIREMENTS

- *Petalinux install requirements*

3.2: VIVADO PROJECT

- *PYNQ-Z2 bdf-files*
- *Digilent vivado library(IP's)*
- *petalinux commands*
- *master XDC(constraints file:line-180:187)*
- *vdma technical document*

4.2 KERNEL CONFIGURATION

- usb-guide
- linux_dma_from_userspace
- how to increase cma size
- linux_reserved memory through dma api
- accessing reserved memory via uio
- xilinx wiki – reserved memory examples

4.4: ADDING USER PACKAGES

- *OpenEmbedded Layer Index*
- *xf86-video-fbdev 0.5.0*
- *libsdl2 2.32.10*

6: LOADABLE KERNEL MODULE

- *kernel modules/drivers creation(visual studio)*
- *loading and creating custom kernel modules/applications using petalinux*
- *vdma_test_src*
- *xilinx_dma_h*
- *xilinx_dma_c*
- *vdma_test_example*
- *client code example frame buffers*
- *ioremap : physical → virt addr space – kernel*
- *kernel dma-egnine api linux*
- *guide explaining linux kernel dma stuff*
- *device tree kernel traversing and getting nodes*
- *fix compiler optimization for kernel driver*

71: ENVIRONMENT SETUP

- *sdk technical manual*

- *cross compilation makefile*